



Figure 6: Selecting the device to run the application

also view many other properties, such as setting size, giving colour, etc, by selecting the 'View all properties' link at the bottom of the *Properties* window.

Coding with Kotlin: The Java/Kotlin part

In normal Java coding, we need the following code to refer to a *TextView*, and for setting a text to the above added *TextView*:

```
TextView t;
t=(TextView) findViewById(R.id.textview);
t.setText("Welcome to OSFY");
```

The above code is used to set a text to the *TextView* element, with the ID 'textview' added to the *layout* file, in Java coding.

In Kotlin, the following code is used:

```
val t: TextView = findViewById(R.id.textview) as TextView;
t.setText("Welcome to OSFY");
```

Here we can see the changes in the coding, while using Java and Kotlin. Also note that if there is any object we declare in the code, which has a null value as default, it will not compile in Kotlin. That is:

```
//Java
String t;
t=null;

//Kotlin
val t: String = null
```

The above code will execute in Java, but will not compile in Kotlin. This is because Kotlin provides 'null safety' or, in other words, Kotlin marks an end to the long-time problem of *NullPointerException*.

Running your first project using Kotlin

In order to run the application, we need to click the *Run* button located on the top panel of Android Studio. A window to choose the device will appear (Figure 6). We can either use an Android virtual device, or an Android device



Figure 7: The output of the program on the device screen

connected to the computer.

If you are using an Android device, then you must enable developer options in it, before enabling *Android Debugging* in the menu options.

Kotlin and Oreo

Android recently announced its next Android version, Android Oreo, along with sparkling new development features. These include downloadable fonts, font support in XML, emoji compatibility, auto sizing *TextView*, and other style and theme options.

What we have just learnt about the features of Kotlin, which is an Android-centric programming language, will help us to work with the new Android features.

For example, let's look at auto-sizing *TextView*. When we code for older versions, prior to Oreo, we need to design separate design files for varying screen sizes, which fit the screen features, like height, width, etc. But in Oreo, we have an auto-sizing option, which will auto-resize the text according to the target device screen. For this, the following code is used:

```
<?xml version="1.0" encoding="utf-8"?>
<TextView
    android:layout_width="match_parent"
    android:autoSizeTextType="uniform" />
```

A new property called *autoSizeTextType* has been introduced in Oreo, which will control the auto sizing of the *TextView*. The value can either be uniform or none, based on the requirement. We have already learnt that connecting elements is easier using Kotlin. This is evident when using the auto-sized *TextView* programmatically with Kotlin, as follows:

```
val t: TextView = findViewById(R.id.textview) as TextView;
t.setText("Welcome to OSFY");
TextViewCompat.setAutoSizeTextTypeWithDefaults(t, TextViewCompat.AUTO_SIZE_TEXT_TYPE_UNIFORM);
```

TextViewCompat.setAutoSizeTextTypeWithDefaults is used for the purpose, where the arguments are *TextView* instant and auto size text type, which can be either *TextViewCompat.AUTO_SIZE_TEXT_TYPE_NONE* or *TextViewCompat.AUTO_SIZE_TEXT_TYPE_UNIFORM*. 

By: Jis Joe Mathew

The author is assistant professor of computer science and engineering at Amal Jyoti College, Kanirapally, Kerala. He can be contacted at jisjoemathew@gmail.com.